Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №1-2

з дисципліни

«Технології створення програмних продуктів»

# Вступ у шаблони проектування. Шаблон Builder та шаблон Factory Method

Група: МІТ-м11

Студент: Переверзев Олексій

Київ 2026

**Завдання 1: Шаблон Factory Method**

Створити програму, використовуючи фабричний метод, яка зможе працювати із декількома типами файлів (текстові, графічні), створюючи бажаний тип файлу за вибором користувача у процесі роботи програми.

Код реалізації:

```
using System;
using System.IO;

namespace FactoryMethodApp
{
    abstract class File
    {
        public abstract void CreatelFile(string fileName);
    }

    class TextFileProduct : File
    {
        public override void CreatelFile(string fileName)
        {
            string fullPath = fileName + ".txt";
            System.IO.File.WriteAllText(fullPath, "Це автоматично згенерований
текстовий файл.");
            Console.WriteLine($"-> Створено текстовий файл за
шляхом:\n{Path.GetFullPath(fullPath)}");
        }
    }

    class GraphicFileProduct : File
    {
        public override void CreatelFile(string fileName)
        {
            string fullPath = fileName + ".png";
            System.IO.File.Create(fullPath).Close();
            Console.WriteLine($"-> Створено графічний файл за
шляхом:\n{Path.GetFullPath(fullPath)}");
        }
    }

    abstract class Creator
    {
        public abstract File FactoryMethod();
    }

    class TextFileCreator : Creator
    {
        public override File FactoryMethod()
        {
            return new TextFileProduct();
        }
    }

    class GraphicFileCreator : Creator
    {
        public override File FactoryMethod()
        {
            return new GraphicFileProduct();
        }
    }
```

```
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Шаблон Factory Method ===");
            Console.WriteLine("Який тип файлу ви хочете фізично створити?");
            Console.WriteLine("1 - Текстовий файл (.txt)");
            Console.WriteLine("2 - Графічний файл (.png)");
            Console.Write("Ваш вибір: ");

            string choice = Console.ReadLine();
            Creator creator = null;

            if (choice == "1")
            {
                creator = new TextFileCreator();
            }
            else if (choice == "2")
            {
                creator = new GraphicFileCreator();
            }
            else
            {
                Console.WriteLine("Невірний вибір!");
                return;
            }

            Console.Write("Введіть ім'я для нового файлу (без розширення): ");
            string fileName = Console.ReadLine();

            File fileObject = creator.FactoryMethod();
            fileObject.CreatelFile(fileName);

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

У цьому завданні застосовано шаблон Factory Method, який визначає інтерфейс для створення об'єкта, але залишає підкласам рішення про те, екземпляр якого класу створювати. Базовий абстрактний клас Creator делегує створення екземплярів файлів підкласам TextFileCreator та GraphicFileCreator. Це дозволило маніпулювати абстрактними об'єктами на високому рівні, замість жорсткої прив'язки до специфічних класів у момент вибору файлу користувачем.

**Завдання 2: Шаблон Builder**

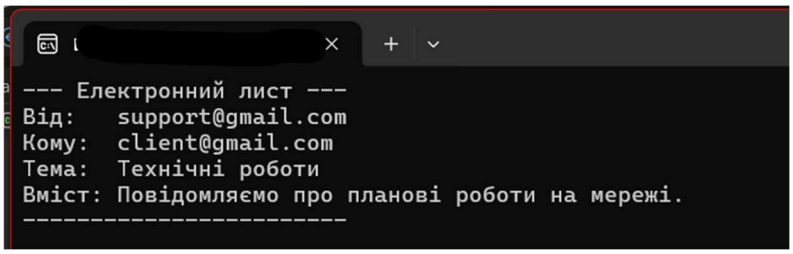
Створити програму MailMessageBuilder, яка буде використовувати шаблон Builder для створення екземпляра електронного листа з можливістю ланцюжкового виклику методів для задання отримувача, відправника, заголовка, вмісту повідомлення.

Код реалізації:

```
using System;

namespace MailMessageBuilderApp
{
    public class MailMessage
    {
        public string Receiver { get; set; }
        public string Sender { get; set; }
        public string Subject { get; set; }
        public string Body { get; set; }

        public void Show()
        {
            Console.WriteLine("--- Електронний лист ---");
            Console.WriteLine($"Від:   {Sender}");
            Console.WriteLine($"Кому:  {Receiver}");
            Console.WriteLine($"Тема:  {Subject}");
            Console.WriteLine($"Вміст: {Body}");
            Console.WriteLine("------------------------\n");
        }
    }

    public abstract class Builder
    {
        public abstract Builder SetSender(string Sender);
        public abstract Builder SetReceiver(string Receiver);
        public abstract Builder SetSubject(string subject);
        public abstract Builder SetBody(string body);
        public abstract MailMessage GetResult();
    }

    public class MailMessageBuilder : Builder
    {
        private MailMessage _email = new MailMessage();

        public override Builder SetSender(string Sender)
        {
            _email.Sender = Sender;
            return this;
        }

        public override Builder SetReceiver(string Receiver)
        {
            _email.Receiver = Receiver;
            return this;
        }

        public override Builder SetSubject(string subject)
        {
            _email.Subject = subject;
            return this;
        }
```

```
        public override Builder SetBody(string body)
        {
            _email.Body = body;
            return this;
        }

        public override MailMessage GetResult()
        {
            return _email;
        }
    }

    public class Director
    {
        public void ConstructSupportEmail(Builder builder)
        {
            builder.SetSender("support@gmail.com")
                   .SetReceiver("client@gmail.com")
                   .SetSubject("Технічні роботи")
                   .SetBody("Повідомляємо про планові роботи на мережі.");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Director director = new Director();
            Builder builder = new MailMessageBuilder();

            director.ConstructSupportEmail(builder);
            MailMessage email1 = builder.GetResult();
            email1.Show();

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:




У завданні реалізовано шаблон Builder, що відокремлює конструювання складного об'єкта (електронного листа) від його подання. Клас Director керує процесом створення, не знаючи кінцевого представлення продукту. Завдяки поверненню поточного об'єкта (return this;) у методах класу MailMessageBuilder, було забезпечено вимогу ланцюжкового виклику методів для гнучкого налаштування параметрів листа.

**Висновки:** під час роботи досліджено шаблони проектування, які допомагають уникати типових помилок та будувати масштабовану архітектуру ПЗ. Factory Method універсалізує код, дозволяючи створювати об'єкти через загальний інтерфейс (Product), коли їхній конкретний тип наперед невідомий. Builder: забезпечує гнучке покрокове конструювання складних об'єктів незалежно від алгоритму їх збірки. Набуті навички є критично важливими для розробки гнучких рішень у реальних інфраструктурах: системах автоматизації, управління мережами та сервісних сповіщень.